

Improving Text Classification with Graph Neural Networks and Transformer Embeddings

Alexander De Jong Roca 294621, Julianna Godziszewska 263841, and
Franciszek Lepka 266618

Wrocław University of Science and Technology

1 Introduction

Recent advances in Transformer-based language models such as BERT have significantly improved performance in many Natural Language Processing (NLP) tasks, including text classification. Transformers model contextual relationships between tokens using attention mechanisms, enabling effective semantic representation learning from large corpora [1, 2]. However, transformer architectures primarily process text as sequences and do not explicitly encode structured linguistic information such as syntactic dependencies between words.

Graph Neural Networks (GNNs) provide a framework for learning from structured relational data by propagating information between connected nodes in a graph. Graph Convolutional Networks (GCNs) learn node representations by iteratively aggregating feature information from neighbouring nodes [3]. In NLP tasks, text can be represented as a graph where words or tokens are nodes and edges correspond to linguistic relationships such as syntactic dependencies or word co-occurrence. Previous research has shown that incorporating graph structure can improve text classification performance in NLP applications [4].

2 Literature Review

2.1 Graph Neural Networks in Natural Language Processing

Traditional Natural Language Processing models, including early neural architectures and sequential models, primarily process text as a one-dimensional sequence of tokens. While this approach is effective for many applications, human language possesses an underlying, multi-dimensional syntactic and logical structure. This structure is often difficult to capture directly through sequential processing, especially when semantic relationships span across long distances in a text. As noted by Wu et al. [4], representing text as a graph allows for the explicit modeling of these complex relationships, laying the foundation for the application of Graph Neural Networks in NLP.

Text-to-Graph Representation Strategies A key engineering challenge in applying GNNs to text is the structural mapping of unstructured language into

a graph topology. The literature highlights several core methodologies. One approach utilizes syntactic graphs, often built from dependency trees. Zhang et al. [5] demonstrated that applying Graph Convolutional Networks (GCNs) over pruned dependency trees effectively filters out linguistic noise, significantly improving relation extraction between distant entities in a sentence.

In the context of document-level tasks like text classification, efficiency and scalability are crucial. Huang et al. [6] introduced the Text-Level Graph Neural Network, which dynamically constructs smaller, individual graphs for single documents rather than a single massive corpus-level graph. This methodology reduces memory overhead and makes inductive learning feasible for production-level classification systems.

Information Propagation and Structural Context The fundamental advantage of GNNs lies in their Message Passing mechanism. Nodes in the text graph (representing words or tokens) iteratively update their feature vectors by aggregating information strictly from their connected neighbors. This allows the model to gain a refined, structure-aware context. The effectiveness of this structural filtering has been proven across diverse NLP tasks. For instance, Bastings et al. [7] utilized GCNs over syntactic dependency trees to improve the encoding of source sentences in neural machine translation, while Peng et al. [8] leveraged graph structures to successfully extract relations that span across multiple consecutive sentences, a task where standard sequential models often fail.

Transition to Hybrid Architectures Given that graph structures excel at enforcing explicit grammatical and logical relationships, researchers recognized the potential of enriching these graphs with high-quality semantic features. Rather than using simple, static word vectors as node features, the integration of deep, contextualized embeddings from advanced language models became the logical next step. This evolution directly leads to the development of hybrid Transformer-GNN architectures, where models like BERT provide robust semantic node representations, and GNNs refine them based on explicit syntactic topology.

Advanced Graph Topologies While syntactic dependency trees provide a strong structural foundation, language often relies on external context and factual relationships not explicitly stated in the text. To address this, researchers have explored integrating Knowledge Graphs directly into the representation space. For example, Liu et al. [9] introduced K-BERT, which injects domain-specific knowledge triples into the sentence structure, allowing the model to make inferences based on external facts rather than just statistical co-occurrence.

Furthermore, single-graph topologies may fail to capture the full complexity of a document. Liu et al. [10] proposed TensorGCN, an architecture that simultaneously constructs semantic, syntactic, and sequential graphs, weaving them into a single tensor for collaborative learning. However, as the complexity and

depth of these graph networks increase, they become susceptible to the over-smoothing problem, where node representations become indistinguishable. As analyzed by Chen et al. [11], carefully managing network depth and topological constraints is a critical engineering requirement when designing deep GNNs for NLP tasks.

2.2 Graph-Based Text Classification Models

To understand the foundations of this approach, the following subsections trace the development of these systems. We begin by reviewing early foundational graph models for text (such as TextGCN), followed by their evolution into hybrid Transformer architectures (like BertGCN), and finally explore specialized applications utilizing explicit syntactic dependency graphs.

Graph-BERT: Only Attention is Needed for Learning Graph Representations Graph-BERT introduces a Transformer-based framework for graph representation learning that eliminates the need for traditional message-passing GNNs. It constructs local subgraphs (e.g., k-hop neighborhoods) and applies attention mechanisms inspired by BERT to model interactions among nodes. Instead of iterative neighborhood aggregation, node representations are learned through attention over sampled substructures.

The key finding is that self-attention can effectively capture graph structure without explicit graph convolution. The model also employs subgraph sampling to improve scalability, demonstrating that attention mechanisms can implicitly encode structural relationships in graphs.

Syntax-Aware Aspect-Level Sentiment Classification with Graph Attention Networks (TD-GAT) This work proposes a graph-based model for sentiment classification that incorporates syntactic dependency structures. Sentences are modeled as graphs where tokens are nodes and dependency relations form edges. A Graph Attention Network (GAT) is used to compute context-aware representations by assigning attention weights to neighboring nodes.

The model leverages syntactic structure to guide information flow while learning task-specific importance over dependencies. When combined with contextual embeddings such as BERT, performance improves further, particularly in capturing long-range dependencies and fine-grained semantic relations.

TextGCN: Graph Convolutional Networks for Text Classification Yao et al. [12] propose TextGCN, the first model to represent an entire text corpus as a single heterogeneous graph containing both word and document nodes. In this graph, edges between word nodes are weighted using positive Pointwise Mutual Information (PMI) computed over sliding windows, whereas edges between word and document nodes are weighted using TF-IDF. The model uses one-hot identity vectors as initial node features, implying that all learned representations emerge entirely from the graph structure.

TextGCN adopts a two-layer Graph Convolutional Network (GCN) following the formulation of Kipf and Welling [3]. In their experiments, TextGCN achieved an accuracy of 86.3% on the 20-Newsgroups dataset and 97.1% on the R8 dataset, outperforming previous bag-of-words approaches. These results demonstrate that global word co-occurrence information encoded in the graph provides a strong signal for text classification.

For our project, TextGCN serves as the baseline based on statistical graph construction. However, its major limitation is that it is a transductive method, requiring access to all documents, including the test set, during training. This limitation motivates our use of an inductive framework based on per-document dependency graphs.

BertGCN: Combining BERT and Graph Convolutional Networks Lin et al. [13] introduce BertGCN, which extends TextGCN by replacing one-hot document features with contextual representations generated by BERT. Specifically, the embedding of the [CLS] token, with dimensionality 768, is used as the feature vector for each document node.

During training, BertGCN maintains a memory bank containing the BERT representations of all documents, which is updated at the end of every epoch. These document embeddings are then passed through the same two-layer GCN architecture used in TextGCN. The final prediction is obtained by linearly interpolating the outputs of the GCN branch and the BERT branch:

$$\hat{y} = \lambda f_{\text{GCN}}(G) + (1 - \lambda) f_{\text{BERT}}(x)$$

where λ is a hyperparameter selected on the validation set.

BertGCN consistently outperforms both standalone BERT and TextGCN across several benchmark datasets, including 20-Newsgroups, R8, R52, Ohsumed, and MR. For instance, on 20-Newsgroups, BertGCN achieves 89.3% accuracy, compared to 85.3% for BERT and 86.3% for TextGCN. These results indicate that the GCN component provides useful smoothing and relational information in the embedding space.

Our project adopts this architecture as a reference baseline. However, instead of using corpus-level co-occurrence edges, we construct graphs from syntactic dependency relations. In contrast to BertGCN, our approach does not use a corpus-level graph or the interpolation mechanism. Instead, we construct independent per-document token-level graphs and use a GAT to propagate BERT token embeddings through the graph structure.

Graph Attention Networks Veličković et al. [14] propose Graph Attention Networks (GAT), which extend GCNs by incorporating an attention mechanism into the neighborhood aggregation process. For a node i and one of its neighbors j , the unnormalized attention score is computed as:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])$$

where $\mathbf{a}$ is a shared attention vector, $\mathbf{W}$ is a linear transformation matrix, and $|$ denotes vector concatenation. The coefficients are then normalized across all neighbors of node i using a softmax function to produce the attention weights α_{ij} .

The model uses multi-head attention, concatenating the outputs of intermediate heads and averaging the heads in the final layer. On standard node classification benchmarks, GAT outperforms GCN. For example, it achieves 83.0% accuracy on Cora and 72.5% on Citeseer, compared with approximately 81.5% and 70.3% for GCN. On the inductive PPI benchmark, GAT reaches 97.3% micro-F1.

Dependency Graph Convolutions for Semantic Role Labeling Marcheggiani and Titov [15] apply GCNs to syntactic dependency graphs for Semantic Role Labeling (SRL). In their approach, each sentence is first parsed into a dependency tree in which every word corresponds to a node. The graph includes directed edges in both directions for every syntactic dependency relation, as well as self-loops on all nodes.

Different edge directions are associated with different parameter matrices. The update rule of the model is defined as:

$$\mathbf{h}_v^{(l+1)} = \text{ReLU} \left(\sum_{u \in \mathcal{N}(v)} \mathbf{W}_{\text{dir}(u,v)}^{(l)} \mathbf{h}_u^{(l)} + \mathbf{b}^{(l)} \right)$$

where $\text{dir}(u, v) \in \{\text{fwd}, \text{bwd}, \text{self}\}$ determines whether the corresponding edge is forward, backward, or a self-loop.

Their model achieves an F1 score of 87.7 on the English CoNLL-2009 benchmark, establishing a new state of the art at the time. Compared with a GCN operating on a fully connected graph, the dependency-based graph improves performance by 2.3 points.

The main conclusion of this work is that sparse syntactic graphs are more effective than dense fully connected graphs because they preserve only linguistically meaningful relations. This observation directly motivates our own graph construction strategy. We therefore use doubly directed dependency edges, self-loops, and different weights depending on edge direction. In addition, following their findings, we restrict the number of GNN layers to one or two in order to avoid over-smoothing.

Syntax-Aware Sentiment Classification Sun et al. [16] propose a syntax-aware model for aspect-level sentiment classification based on graph convolutions over dependency trees. Their architecture first applies a BiLSTM to produce contextualized word representations and then performs graph convolution over the dependency tree of the sentence. This allows sentiment information to propagate efficiently from opinion words to the target aspect.

The model achieves state-of-the-art results on the SemEval aspect-based sentiment analysis benchmarks, obtaining approximately 80% accuracy on restaurant reviews. Similarly, Huang and Carley [17] combine BERT with an aspect-directed Graph Attention Network, producing around 80% accuracy on the laptop benchmark and 83% on the restaurant benchmark.

Both studies observe that the dependency path between an opinion word and an aspect term is usually very short, typically between one and three hops. Consequently, only a small number of GCN or GAT layers is required. Furthermore, the ablation experiments of Huang and Carley [17] show that replacing dependency edges with a fully connected graph decreases accuracy by approximately 1–1.3 percentage points.

These results provide strong evidence that syntactic dependency structures contain valuable information that is not fully captured by sequential models alone. Our proposed system follows the same principle by combining BERT embeddings with a graph neural network operating on a dependency graph extracted using spaCy. The overall processing pipeline can be summarized as:

$$\text{BERT} \rightarrow \text{Dependency Graph} \rightarrow \text{GNN} \rightarrow \text{Classifier}$$

3 Methodology

3.1 Project Idea Definition

Our project proposes a hybrid architecture that combines the deep semantic representation power of Pre-trained Language Models (specifically BERT) with the relational inductive bias of Graph Neural Networks (GNNs).

Instead of constructing a single corpus-level graph where each node represents one document, we construct independent document-level graphs. In this setting, each document is represented as a token-level graph, where nodes correspond to token occurrences and edges represent relationships between tokens. This allows the model to capture structural relationships inside each document, such as local co-occurrence patterns or syntactic dependency relations.

The project is organized in two steps. First, we isolate the effect of graph topology by comparing statistical co-occurrence graphs with syntactic dependency graphs under the same GAT architecture. In this first experiment, node features are initialized using trainable word embeddings rather than BERT embeddings, so that the effect of graph construction can be analyzed independently from contextual Transformer representations.

Second, we evaluate the full hybrid architecture. In this setting, BERT provides contextual token embeddings, and the graph neural network propagates information through the selected graph structure to improve classification performance. This formulation enables an inductive pipeline, where unseen documents can be processed as independent graphs without reconstructing a global corpus-level graph.

3.2 Graph Construction Pipelines

We propose to construct document-level token graphs. Let $X = \{x_1, x_2, \dots, x_n\}$ denote the set of documents, and let each document x_i be represented as a sequence of tokens:

$$x_i = (t_1, t_2, \dots, t_m)$$

where each token occurrence t_j corresponds to one node in the graph.

Embedding Extraction In Experiment 1, each token is encoded using a trainable embedding layer. This ensures that both graph variants use the same node feature initialization, allowing us to isolate the effect of graph topology. For computational efficiency, documents are truncated to a maximum of 256 tokens after preprocessing.

In the hybrid experiment, each document is encoded using a pre-trained BERT model. Token-level contextual embeddings are extracted from BERT and used as node features in the graph:

$$h_j \in R^d$$

where h_j represents the embedding of token t_j .

Statistical Co-occurrence Graph Construction We construct a graph $G = (V, E)$ where each node corresponds to a token occurrence in a document. In the statistical co-occurrence graph, edges are created based on local proximity in the token sequence. If two tokens appear within the same fixed-size sliding window, their corresponding nodes are connected.

The graph is treated as undirected, but it is implemented using two directed edges:

$$(t_i, t_j) \in E \quad \text{and} \quad (t_j, t_i) \in E$$

Self-loops are also added to all nodes. This graph captures local surface-level relationships between nearby tokens.

Syntactic Dependency Graph Construction We also construct a syntactic dependency graph using the spaCy parser. In this graph, edges are created according to grammatical dependency relations between tokens. Each token is connected to its syntactic head in the dependency tree.

Dependency edges are made bidirectional, allowing information to flow both from heads to dependents and from dependents to heads. Self-loops are also added to all nodes. This graph captures grammatical relationships between tokens, including relations between words that are not necessarily adjacent in the original sentence.

Graph Variants To evaluate the impact of graph construction, we compare two graph variants: statistical co-occurrence graphs and syntactic dependency graphs. The goal of this comparison is to isolate the effect of graph topology while keeping the model architecture, node feature initialization, and training procedure fixed.

Scalability Considerations Since each document is represented as an independent graph, the model does not require constructing a single large corpus-level adjacency matrix. This reduces memory requirements and makes the approach inductive, since unseen test documents can be converted into graphs and classified independently.

3.3 Model Architectures

The constructed graph is used as input to a Graph Neural Network, where node features are initialized with trainable word embeddings in Experiment 1 and with BERT token embeddings in the hybrid experiment.

Baseline BERT-only: As a baseline for the hybrid experiment, we employ a standard text classification approach in which each document is independently encoded using BERT. The resulting representation is then passed to a lightweight classifier, such as a multi-layer perceptron, to predict the corresponding class labels.

This approach does not incorporate any explicit graph structure and serves as a reference point for evaluating the contribution of graph-based methods.

GAT-based Model: In the GAT-based model, we utilize the constructed graph structure to enable information exchange between related tokens. Each node in the graph is initialized with its corresponding token embedding. During training, the GAT updates these representations by aggregating information from neighboring nodes using attention weights.

This process allows each document representation to be enriched with structural information from its token-level graph. After message passing, the updated node representations are aggregated via global mean pooling into a single document-level representation, which is passed to the classifier.

Hybrid BERT-GAT Model: In the hybrid model, BERT is used to obtain contextual token embeddings for each document. These embeddings are assigned to the corresponding token nodes in the selected graph topology. The GAT then refines the token representations by propagating information through the graph edges. Finally, the updated node representations are pooled into a document-level representation and passed to the classifier.

This model allows us to evaluate whether explicit graph structure provides additional information beyond the contextual representations learned by BERT.

4 Experimental Design

To rigorously evaluate the proposed approaches, we designed a structured experimental protocol. This section defines the specific research questions, the corresponding experiments, and the configuration of the research environment.

4.1 Research Questions

We formulate three research questions to examine different properties of our data topologies and neural network methods:

1. **Graph Topology:** How do linguistically motivated syntactic dependency graphs compare to statistical word co-occurrence networks regarding text classification accuracy and macro-F1?
2. **Integration Impact:** To what extent does the integration of syntactic structures via GNNs improve classification performance compared to purely sequential semantic representations generated by BERT model?
3. **Scalability:** How does the hybrid model’s performance scale with increasing size of a dataset?

4.2 Experiment 1: Influence of Graph Topology

Experiment Overview We designed this experiment to isolate how graph topology affects text classification. We skipped BERT embeddings entirely to focus purely on topology before moving to the full hybrid Transformer-GNN architecture. Both graph variants run through the identical Graph Attention Network (GAT) with trainable word embeddings. With the architecture, node features, training, and evaluation locked in, we can test just the graph construction method.

We compared statistical co-occurrence graphs with syntactic dependency graphs. Each document becomes an independent token-level graph where nodes map to token occurrences. The only real difference between the two approaches is how they draw the edges.

This experiment addresses the following research question:

How does intra-document graph topology, syntactic dependency edges versus sliding-window co-occurrence edges, affect text classification accuracy and macro-F1?

We evaluate the two graph topologies on two text classification datasets: the IMDb Movie Reviews dataset for binary sentiment classification and a scientific text classification dataset for multi-class topic classification. Using two datasets allows us to see the consistency of the results in various types of texts

Graph Representation Each document is represented as an independent token-level graph. Given a document tokenized into a sequence $x = (t_1, t_2, \dots, t_m)$, each token occurrence t_i is assigned a dedicated node v_i . Repeated words at different positions are treated as distinct nodes, preserving the local sequential structure of the document and enabling inductive classification of unseen documents without a global corpus-level graph. Self-loops are added to every node in both graph variants to ensure each token retains its own representation during message passing.

Statistical Co-occurrence Graph The first topology we are using is a statistical co-occurrence graph. The edges are defined using local proximity in token sequence: each token t_i is connected to every token within a positional distance of at most 3 in either direction, creating a neighbourhood of up to 6 tokens per node. All edges are unweighted and bidirectional, implemented as two directed edges (v_i, v_j) and (v_j, v_i) for every pair that is connected. This topology captures surface lexical proximity without relying on linguistic parsing, and its used as the statistical baseline for our comparison

Syntactic Dependency Graph The second topology is a syntactic dependency graph constructed using the spaCy parser. Each token is connected to its syntactic head according to the sentence dependency tree. As in the co-occurrence graph, all dependency edges are bidirectional, allowing information to propagate both from heads to dependents and from dependents to heads. Unlike co-occurrence edges, dependency edges don't have a physical distance constraint: they connect grammatically related tokens regardless of their position in the sequence.

Node Features In this experiment, all nodes are initialised with trainable word embeddings. Each token is mapped to a vocabulary index built exclusively from the training split, and an embedding layer learns a dense vector representation during training. This choice is deliberate: by keeping node features identical across both graph variants, the only experimental variable is the edge construction method, allowing us to attribute any performance differences directly to graph topology.

Model Architecture Both graph topologies are processed by the same two-layer Graph Attention Network (GAT) [14]. Token IDs are first mapped to dense vectors via the trainable embedding layer. The resulting node features are then refined by two successive GATConv layers: the first applies multi-head attention (4 heads, outputs concatenated), and the second reduces the representation to a fixed hidden dimension (1 head). After message passing, node representations are aggregated into a single document embedding via global mean pooling, which is subsequently passed to a linear classifier. Dropout is applied after each GAT layer to regularise training.

Training Procedure Models are trained under a supervised, inductive setting: each document graph is processed independently, so the model can classify unseen documents without accessing the full training corpus. The dataset is partitioned into training, validation, and test sets (70%/15%/15%) using stratified sampling to preserve class distributions. Both models are optimised with AdamW and trained for up to 30 epochs, with early stopping based on validation Macro-F1 (patience = 5). Key hyperparameters are shared across both conditions: embedding dimension 128, hidden dimension 64, 4 attention heads,

dropout 0.3, learning rate 10^{-3} , and weight decay 10^{-4} . To account for training stochasticity, all experiments are repeated with five random seeds (13, 21, 42, 87, 100).

Evaluation Protocol We evaluate models on the held-out test set using **Accuracy** (overall proportion of correctly classified documents) and **Macro-F1** (which weights all classes equally, making it more informative in multi-class settings). Results are reported as mean $\pm$ standard deviation across five seeds. We use a paired Student’s t -test ($\alpha = 0.05$) to check whether differences between topologies are statistically significant. Classification reports and confusion matrices from the final seed run (seed = 100) are included as qualitative diagnostics only.

Expected Outcome We do not assume either topology will win. A statistically significant edge for the dependency graph would mean explicit syntactic structure adds something beyond local word proximity. If the co-occurrence graph performs comparably, surface-level proximity is probably enough under this setup. When no significant difference appears, we choose the topology for the next experiment based on mean validation Macro-F1, computational cost, or linguistic interpretability.

Limitations Co-occurrence graphs produce roughly 2.4 times more edges than dependency graphs on average. This density gap may independently affect how information aggregates during message passing, which partially confounds the topology comparison. We also use trainable word embeddings rather than contextual BERT representations. This is intentional since we want to isolate topological effects, but it means the numbers here are not representative of the full hybrid BERT-GAT architecture tested in Experiment 2.

4.3 Experiment 2: Integration Impact

Experiment Overview We compare a purely sequential Transformer (standalone BERT) against a hybrid architecture that combines BERT embeddings with the best-performing graph topology from Experiment 1. We use the same two datasets throughout: Scientific Text Classification and IMDb Movie Reviews.

Architectures Compared

- **Standalone BERT:** Raw text goes through a pre-trained BERT model, which relies entirely on its internal self-attention to capture relationships between words. The final prediction comes from passing the [CLS] token embedding through a linear classification layer.

- **Hybrid Architecture (BERT + Graph):** Text goes through two pipelines at once. BERT extracts a contextualized embedding for every token. In parallel, the text is parsed into the sparse graph from Experiment 1, with the BERT embeddings assigned as initial node features. A GAT updates those representations by passing messages along the graph edges. The resulting node vectors are pooled into a single document representation and passed to the classifier.

Training Process Both architectures start from the same pre-trained weights (e.g., bert-base-uncased). For the baseline, we fine-tune the BERT layers and classification head directly. For the hybrid, the BERT weights initialize the node features, while the GAT layers and classifier are trained from scratch. Both models use the AdamW optimizer with the same batch sizes and early stopping criteria as Experiment 1.

Testing and Evaluation We evaluate both models on the held-out test sets using Accuracy and Macro-F1. The comparison shows whether adding an explicit sparse graph structure on top of BERT’s self-attention improves classification. A paired Student’s t -test ($\alpha = 0.05$) tells us whether any observed difference is statistically significant.

4.4 Experiment 3: Scalability and Data Efficiency

Experiment Overview This experiment addresses RQ3 by investigating how the size of the training dataset affects the learning curve of the models. The objective is to measure how quickly and effectively a model learns when training examples are severely limited. To keep the analysis focused and clear, we conduct this experiment exclusively on the IMDb Movie Reviews dataset.

Methodology and Data Splitting Instead of training the models on the entire available dataset (as done in previous experiments), we artificially restrict the amount of training data. We partition the training set into incremental fractions: 10%, 25%, 50%, and 100% of the total available samples. We then independently train both the Standalone BERT baseline and our Hybrid Architecture on each of these subsets. Regardless of how much training data was used, every model is evaluated against the exact same, full-sized test set to ensure a fair comparison.

Evaluation We record the Accuracy and Macro-F1 scores for each model at every data fraction, as well as the training convergence rate (e.g., the number of epochs required to reach optimal performance). The primary objective is to investigate whether incorporating explicit grammatical structures via GNNs acts as a strong inductive bias that accelerates the learning process.

4.5 Implementation

We wrote the experimental code in Python, building the neural networks with PyTorch and PyTorch Geometric (PyG). We loaded the contextual word embeddings and pre-trained Transformer weights through the HuggingFace `transformers` library.

We had to reconcile the different tokenization strategies between the models. The `spaCy` dependency parser splits text into whole linguistic words to form the graph nodes, while BERT uses a WordPiece tokenizer that often breaks complex words into smaller sub-tokens. To map BERT’s outputs to the graph structure, we wrote a sub-word averaging function. Whenever BERT split a word, we averaged the embeddings of its sub-tokens to create a single node feature vector for the GAT layers. Project code is available at: <https://github.com/alexdejongr/NLP-project>.

5 Evaluation and Results Analysis

5.1 Experiment 1: Influence of Graph Topology

Experiment 1 isolates how different graph construction strategies affect document-level text classification. We compare two intra-document topologies: statistical co-occurrence graphs (using a fixed sliding window of size 3) and syntactic dependency graphs (extracted with `spaCy`). We kept the documents, vocabulary, trainable word embeddings, and Graph Attention Network (GAT) architecture identical for both variants. This means the graph topology, represented by the edges used during message passing, is the only independent variable.

Table 1. Classification performance in Experiment 1, reported as mean $\pm$ standard deviation across 5 random seeds.

Dataset	Graph Topology	Accuracy (%)	Macro-F1 (%)
Scientific Text	Co-occurrence	66.92 $\pm$ 3.22	66.77 $\pm$ 3.33
Scientific Text	Dependency	65.33 $\pm$ 2.66	65.08 $\pm$ 2.78
IMDb Reviews	Co-occurrence	74.60 $\pm$ 1.67	74.43 $\pm$ 1.72
IMDb Reviews	Dependency	76.47 $\pm$ 1.12	76.40 $\pm$ 1.16

Table 1 shows the accuracy and Macro-F1 scores obtained by both graph topologies on the Scientific Text and IMDb Reviews datasets. On the Scientific Text dataset, the co-occurrence graph obtains slightly higher mean performance than the dependency graph. In contrast, on the IMDb Reviews dataset, the dependency graph obtains slightly higher mean performance than the co-occurrence graph. However, these differences must be interpreted together with the statistical significance analysis.

To determine whether the observed differences between the two graph construction methods are statistically significant, we conducted a paired Student’s t-test with $\alpha = 0.05$ across the 5 random seeds.

Table 2. Paired Student’s t-test p-values for Experiment 1 ($\alpha = 0.05$).

Dataset	Accuracy p-value	Macro-F1 p-value	Statistically Significant?
Scientific Text	0.3537	0.3403	No
IMDb Reviews	0.1719	0.1679	No

As shown in Table 2, all p-values are above the $\alpha = 0.05$ threshold. Therefore, the differences between co-occurrence and dependency graphs are not statistically significant for either dataset. Although the co-occurrence graph achieves slightly better average results on Scientific Text and the dependency graph achieves slightly better average results on IMDb Reviews, the statistical test does not provide sufficient evidence to conclude that one graph topology consistently outperforms the other.

Regarding Research Question 1, which investigates whether syntactic dependency graphs provide a measurable advantage over statistical co-occurrence graphs for text classification, the results do not support a strong conclusion in favor of either topology. Under the current GAT setup, graph topology alone does not show a statistically significant effect on classification performance. This suggests that both local surface-level proximity and syntactic dependency relations can provide useful structural information, but neither approach is clearly superior in this isolated experiment.

Several limitations must be considered when interpreting these findings. First, Experiment 1 uses trainable word embeddings initialized from scratch rather than contextual BERT embeddings. This choice is intentional, because the experiment is designed to isolate the effect of graph topology. However, these results should not be interpreted as the final performance of the full hybrid BERT-GAT architecture. Second, the two graph construction methods differ substantially in edge density. In the Scientific Text dataset, the co-occurrence graphs contain on average 1092.29 edges per training graph, compared to 460.32 edges for dependency graphs, resulting in an edge density ratio of 2.37. In the IMDb Reviews dataset, co-occurrence graphs contain on average 1252.94 edges, compared to 522.11 edges for dependency graphs, resulting in an edge density ratio of 2.40. Therefore, performance differences may be influenced not only by the linguistic nature of the edges, but also by graph sparsity and neighborhood size. Finally, the classification reports and confusion matrices are based on the last seed (seed = 100) and are used only as diagnostic tools; individual-seed results may differ from the reported means, while the main conclusions are based on the averaged results across all five seeds.

5.2 Experiment 2: Integration Impact

Since Experiment 1 found no statistically significant difference between the two topologies, we select the syntactic dependency graph for Experiment 2, as it is more linguistically motivated and aligns with the paper’s research focus on structural grammatical information.

To see how much the graph helps, we tested two versions of our hybrid model: a “frozen” variant (where BERT’s weights are locked, and only the graph learns) and an “end-to-end” variant (where BERT and the graph learn together). All models were trained for a duration of 5 epochs.

Table 3 shows the classification results. The Standalone BERT baseline achieved the highest scores. The frozen BERT+GAT model performed the worst, but the end-to-end variant improved significantly, almost catching up to the baseline (e.g., 89.04% vs. 89.59% on the IMDb dataset).

Table 3. Macro-F1 scores (mean $\pm$ std over 5 seeds) for Experiment 2.

Architecture	Scientific Text	IMDb Movie Reviews
BERT + GAT (frozen)	76.48 $\pm$ 0.72%	86.46 $\pm$ 0.69%
BERT + GAT (end-to-end)	79.15 $\pm$ 2.71%	89.04 $\pm$ 1.12%
Standalone BERT	79.63 $\pm$ 1.00%	89.59 $\pm$ 0.72%

To ensure a fair test, the dataset splits (70/15/15) were fixed with a random seed, meaning all models saw the exact same test data. Even with this setup, the BERT+GAT model did not beat standalone BERT. We identified three potential reasons for this:

- **BERT already understands syntax:** BERT is trained on huge amounts of text. Its internal attention mechanism already learns to recognize grammatical rules and word connections. By adding a graph on top, we are mostly just repeating information that BERT already knows.
- **Throwing away the [CLS] token:** Standalone BERT classifies text using the special [CLS] token, which is heavily trained to summarize the whole document. Our graph model ignores this token and instead calculates an average of all word vectors (global mean pooling). This is a weaker way to summarize a text, which explains a large part of the performance drop.
- **The graph sees less text:** To build the graph, the spaCy parser had to cut the text at 256 tokens. Standalone BERT, however, can read up to 512 tokens directly. For long texts like IMDb reviews, the graph model simply had less information to work with.

In conclusion, the large improvement in the end-to-end variant shows that freezing BERT was the main problem. However, explicitly adding a dependency graph does not give the model an advantage over what BERT can already do on its own using self-attention.

5.3 Experiment 3: Scalability and Data Efficiency

The objective of Experiment 3 is to investigate how the amount of available training data affects the performance of the standalone BERT baseline and the hybrid BERT-GAT model, in order to address Research Question 3. Following the findings of Experiment 1, the hybrid model uses the syntactic dependency graph topology. We conduct this experiment exclusively on the IMDb Movie Reviews dataset. The training split is restricted to incremental fractions of 10%, 25%, 50%, and 100%, corresponding to 140, 350, 700, and 1400 documents respectively, while both models are always evaluated on the same full-sized test set to ensure a fair comparison. In addition to Accuracy and Macro-F1, we record the number of epochs required to reach the best validation Macro-F1 as a coarse indicator of convergence rate. All results are reported as mean $\pm$ standard deviation across three random seeds (13, 21, 42).

Table 4. Classification performance in Experiment 3 on the IMDb Reviews dataset, across increasing fractions of the training set, reported as mean $\pm$ standard deviation over three random seeds. “Epochs” denotes the epoch at which the best validation Macro-F1 was reached.

Fraction	Model	Accuracy (%)	Macro-F1 (%)	Epochs
10%	BERT	83.67 $\pm$ 1.33	83.66 $\pm$ 1.34	7.0 $\pm$ 3.6
10%	BERT-GAT	85.56 $\pm$ 0.51	85.55 $\pm$ 0.50	5.0 $\pm$ 1.0
25%	BERT	86.56 $\pm$ 1.02	86.55 $\pm$ 1.03	2.0 $\pm$ 1.0
25%	BERT-GAT	89.11 $\pm$ 0.51	89.10 $\pm$ 0.51	4.3 $\pm$ 2.3
50%	BERT	87.00 $\pm$ 1.20	86.98 $\pm$ 1.19	3.7 $\pm$ 2.3
50%	BERT-GAT	89.33 $\pm$ 1.33	89.31 $\pm$ 1.35	2.0 $\pm$ 1.0
100%	BERT	88.78 $\pm$ 1.68	88.77 $\pm$ 1.68	2.0 $\pm$ 1.0
100%	BERT-GAT	88.22 $\pm$ 2.22	88.21 $\pm$ 2.22	3.3 $\pm$ 2.5

Table 4 shows the Accuracy and Macro-F1 scores obtained by both architectures at each training fraction. In the low- and medium-data regimes, the hybrid BERT-GAT model consistently outperforms the standalone baseline. The Macro-F1 advantage of BERT-GAT over BERT is +1.89 points at 10% of the data, +2.55 points at 25%, and +2.33 points at 50%. At the 25% and 50% fractions this advantage exceeds the standard deviation of either model across seeds, indicating that the improvement is consistent rather than an artefact of a single favourable run. This pattern is consistent with the hypothesis that the explicit syntactic dependency structure acts as an inductive bias that is most valuable when training examples are scarce, allowing the hybrid model to generalise from fewer labelled documents than the purely sequential Transformer.

The advantage of the hybrid model diminishes as more data becomes available. At 100% of the training set the two architectures are effectively tied, with BERT reaching a Macro-F1 of 88.77% and BERT-GAT reaching 88.21%, a difference of only -0.56 points. This gap is well within one standard deviation of either model and is therefore not meaningful: once the full training set is avail-

able, the explicit graph structure no longer provides a measurable benefit over the standalone Transformer. As illustrated by the learning curves in Figure ??, the two models start clearly separated in the low-data regime and converge as the dataset grows, meeting near the full-data point. This suggests that when sufficient labelled data is present, BERT’s self-attention is able to capture the relevant relational structure on its own, making the additional graph component redundant. The epochs-to-best column, used here only as a coarse indicator of convergence rate, does not reveal a clear or monotonic difference between the two architectures and should be interpreted with caution, as the standard deviations are large relative to the differences in means.

Regarding Research Question 3, which investigates how the hybrid model’s performance scales with the size of the training set, these results indicate that the contribution of the syntactic dependency graph is strongest in data-limited settings and is gradually absorbed by the Transformer as the dataset grows, rather than scaling uniformly with dataset size. The dependency structure appears to compensate for limited supervision, and its advantage vanishes once enough training data is available.

6 Conclusion

Table 2 shows that all p-values remain above the $\alpha = 0.05$ threshold, meaning the differences between co-occurrence and dependency graphs are not statistically significant for either dataset. Co-occurrence graphs average slightly better on Scientific Text, and dependency graphs average slightly better on IMDb Reviews, but neither topology consistently outperforms the other.

Research Question 1 asks whether syntactic dependency graphs provide a measurable advantage over statistical co-occurrence graphs. In this isolated experiment, they do not. Under the current GAT setup, graph topology alone did not strongly affect classification performance. Both local surface-level proximity and syntactic dependency relations capture structural information, but neither proved superior.

We intentionally initialized the word embeddings from scratch to isolate the effect of the graph topology. Because we did not use contextual BERT embeddings, these results do not reflect the performance of the full hybrid BERT-GAT architecture.

The two construction methods also produce very different edge densities. In the Scientific Text dataset, co-occurrence graphs average 1092.29 edges per training graph compared to 460.32 for dependency graphs (a ratio of 2.37). In the IMDb Reviews dataset, they average 1252.94 edges compared to 522.11 (a ratio of 2.40). Neighborhood size and graph sparsity likely influence performance just as much as the linguistic nature of the edges.

We report classification metrics and confusion matrices for the final seed (seed = 100) as diagnostic examples. Because individual runs vary, our main conclusions rely entirely on the five-seed averages.

In Experiment 2, we evaluated the integration impact by comparing a purely sequential Transformer against our proposed hybrid architecture (BERT-GAT). The hybrid model utilized the syntactic dependency graph selected in Experiment 1 and was tested in both frozen and end-to-end training configurations to isolate the contribution of the explicit graph structure.

The results indicate that explicitly incorporating grammatical structure via GNNs does not improve overall classification performance compared to the standalone baseline. Standalone BERT consistently outperformed both hybrid variants across the datasets. While the end-to-end hybrid model significantly narrowed the performance gap compared to the frozen variant, what confirmed that freezing BERT’s weights was a major bottleneck, it still did not surpass the baseline’s Accuracy and Macro-F1 scores.

The results show that BERT’s standard pre-training is already sufficient for document-level text classification. Its internal self-attention mechanism successfully captures the grammar and meaning of the text, allowing it to achieve the best results on its own. Therefore, adding a complex graph structure on top of BERT is unnecessary and does not improve performance in general document classification.

In Experiment 3, we examined scalability and data efficiency by training the standalone BERT baseline and the hybrid BERT-GAT model on increasing fractions of the IMDB training set (10%, 25%, 50%, and 100%), always evaluating on the same full-sized test set. This experiment was designed to test whether the explicit grammatical structure of the graph acts as an inductive bias that helps most when labelled data is scarce.

The results reveal a clear data-dependent pattern. In the low- and medium-data regimes, the hybrid model consistently outperformed the standalone baseline, with Macro-F1 advantages of +1.89, +2.55, and +2.33 points at 10%, 25%, and 50% of the data respectively; at the 25% and 50% fractions this advantage exceeded the standard deviation of either model across seeds. At the full data fraction, however, the two architectures were effectively tied, differing by only -0.56 Macro-F1 points, a gap well within one standard deviation. The explicit dependency structure therefore appears to compensate for limited supervision, but its benefit is gradually absorbed by the Transformer as the dataset grows.

This finding refines the conclusion drawn from Experiment 2. The observation that standalone BERT is sufficient for document-level classification holds primarily in the full-data setting; when training data is limited, the syntactic graph still provides a measurable and consistent advantage. In other words, the value of explicitly incorporating grammatical structure is not fixed but depends on data availability, being most useful exactly when labelled examples are few.

This also carries a practical implication. Because the hybrid model reaches most of its achievable performance early in the data regime, training on a reduced subset can save substantial parsing and training time at little cost: BERT-GAT trained on only 25% of the data (350 documents) reached a Macro-F1 of 89.10%, comparable to the 88.21% obtained on the full 1400-document set.

Bibliography

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention Is All You Need. *Advances in Neural Information Processing Systems* (2017). Available at: <https://arxiv.org/abs/1706.03762>
- [2] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT* (2019). Available at: <https://arxiv.org/abs/1810.04805>
- [3] Kipf, T.N., Welling, M.: Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations* (2017). Available at: <https://arxiv.org/abs/1609.02907>
- [4] Wu, L., Cui, P., Pei, J., Zhao, L.: Graph Neural Networks for Natural Language Processing: A Survey. *Foundations and Trends in Machine Learning* (2021). Available at: <https://arxiv.org/abs/2106.06090>
- [5] Zhang, Y., Qi, P., Manning, C.D.: Graph Convolution over Pruned Dependency Trees Improves Relation Extraction. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2205–2215 (2018). Available at: <https://aclanthology.org/D18-1244/>
- [6] Huang, L., Ma, D., Li, S., Zhang, X., Wang, H.: Text Level Graph Neural Network for Text Classification. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3444–3450 (2019). Available at: <https://aclanthology.org/D19-1345/>
- [7] Bastings, J., Titov, I., Aziz, W., Marcheggiani, D., Sima'an, K.: Graph Convolutional Encoders for Syntax-aware Neural Machine Translation. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 1957–1967 (2017). Available at: <https://aclanthology.org/D17-1209/>
- [8] Peng, N., Poon, H., Quirk, C., Toutanova, K., Yih, W.-t.: Cross-Sentence N-ary Relation Extraction with Graph LSTMs. *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 101–115 (2017). Available at: <https://aclanthology.org/Q17-1008/>
- [9] Liu, W., Zhou, P., Zhao, Z., Wang, Z., Ju, Q., Deng, H., Wang, P.: K-BERT: Enabling Language Representation with Knowledge Graph. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 03, pp. 2901–2908 (2020). Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/5681>
- [10] Liu, X., You, X., Zhang, X., Wu, J., Lv, P.: TensorGCN: Collaborative Graph Convolutional Networks for Text Classification. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, pp. 8409–8416 (2020). Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/6359>
- [11] Chen, D., Lin, Y., Li, W., Li, P., Zhou, J., Sun, X.: Measuring and Relieving the Over-smoothing Problem for Graph Neural Networks from the

- Topological View. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 3438–3445 (2020). Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/5747>
- [12] Yao, L., Mao, C., Luo, Y.: Graph Convolutional Networks for Text Classification. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 1, pp. 7370–7377 (2019). Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/4695>
- [13] Lin, Y., Meng, Y., Sun, X., Han, Q., Kuang, K., Li, J., Wu, F.: Bert-GCN: Transductive Text Classification by Combining GCN and BERT. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 1456–1462 (2021). Available at: <https://aclanthology.org/2021.findings-acl.126/>
- [14] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph Attention Networks. In: *International Conference on Learning Representations (ICLR)* (2018). Available at: <https://arxiv.org/abs/1710.10903>
- [15] Marcheggiani, D., Titov, I.: Encoding Sentences with Graph Convolutional Networks for Semantic Role Labeling. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 1506–1515 (2017). Available at: <https://aclanthology.org/D17-1159/>
- [16] Sun, K., Zhang, R., Mensah, S., Mao, Y., Liu, X.: Aspect-Level Sentiment Analysis via Convolution over Dependency Tree. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 5679–5688 (2019). Available at: <https://aclanthology.org/D19-1569/>
- [17] Huang, B., Carley, K.M.: Syntax-Aware Aspect Level Sentiment Classification with Graph Attention Networks and BERT. arXiv preprint arXiv:1909.02606 (2019). Available at: <https://arxiv.org/abs/1909.02606>